

SWE MULTI-STEP Long Horizon Engineering Tasks

Long-horizon structure, language selection, deterministic reward, and optional RewardKit judges

Important links:

- [+ SWE_multistep_long_horizon_task_review_checklist](#)

Jibble:

Jibble Activity: SWEBench-Long-Horizon_Internal-Research_Saurabh

Jibble Project: Google_SuperRater_STEM

DataOS Dataset: <https://data-os.turing.com/datasets/169>

1. What This Task Is

A Tier 2 repo-engineering task is one real software change carried out over several stages. The agent works in the complete repository. The code and services stay in place between stages, but the agent receives a fresh briefing at each stage.

The task is meant to feel like real engineering work. First the agent finds and proves the problem. Then it implements the main fix. Next it receives review or CI feedback. Finally it handles faults, performance, recovery and cleanup.

The simple rule

Code must pass executable checks at every stage. The selected RewardKit judge can review process and quality, but it cannot turn broken behavior into a pass.

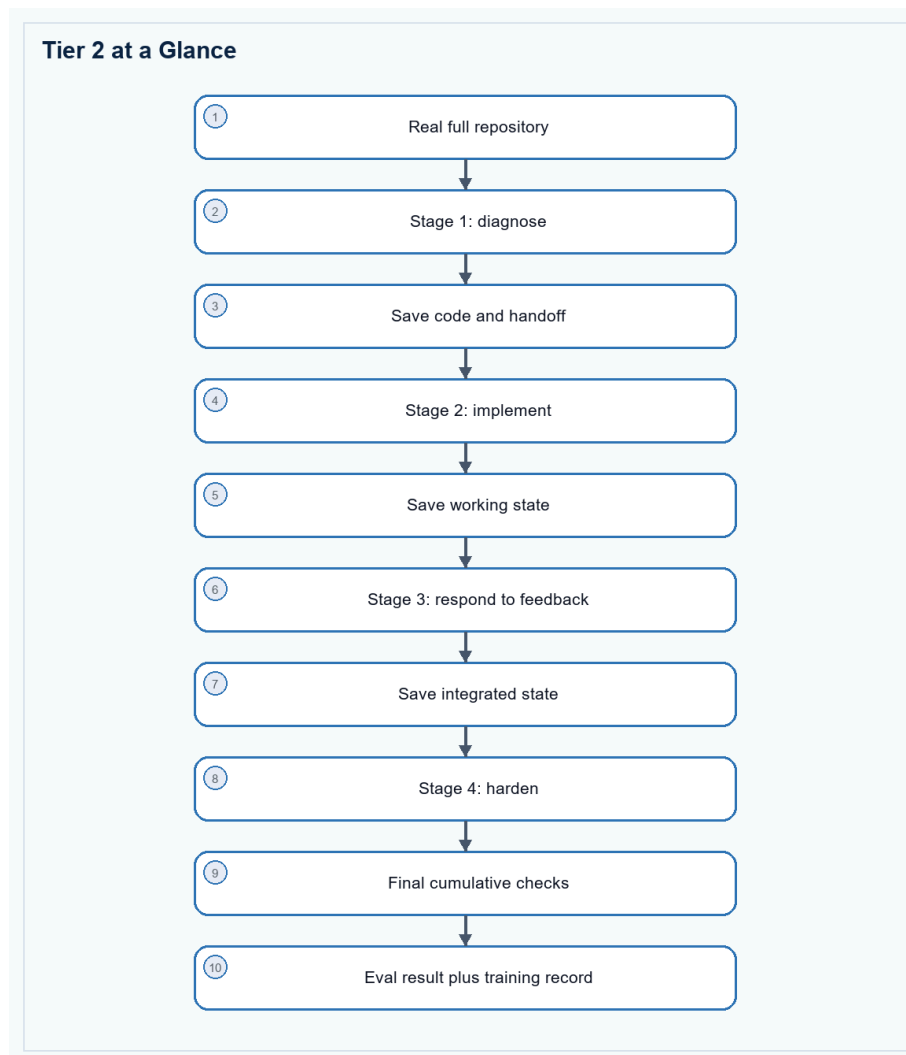


Figure 1. One repository moves through four connected engineering stages.

Tier 2 property	What it means
Full repository	The agent can inspect and change the real project, not a toy extract.
Persistent state	Code, services and named artifacts survive between stages.
Fresh stage context	Each stage gives only the new briefing and approved handoff.
New evidence	Later stages reveal review, CI, runtime, fault or performance information.
Strict stage gates	A stage advances only when its required executable checks pass.
Training value	The run keeps stage-by-stage evidence, not only the final patch.

What Makes It Long Horizon

A long horizon does not simply mean a large patch or a long timeout. It means the work develops over time. Earlier decisions affect later stages, new evidence appears after work has already been done, and the final result must remain correct as the task becomes harder.

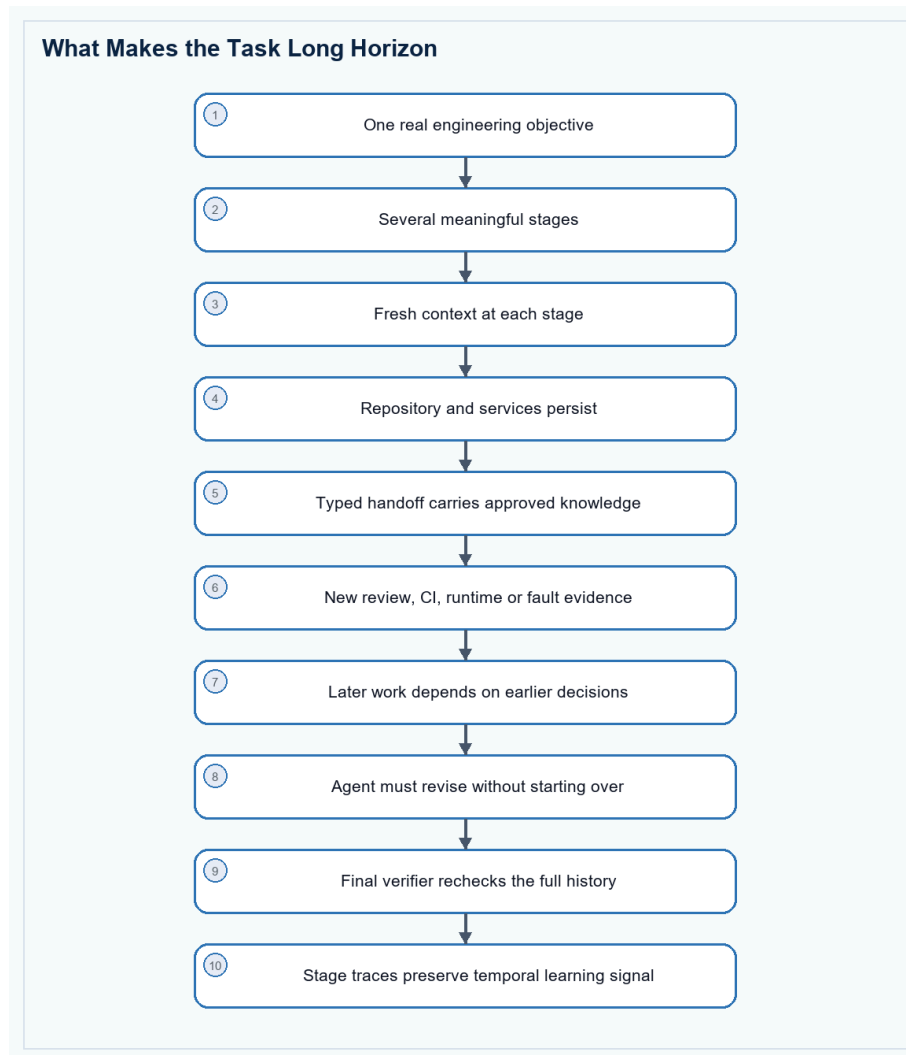


Figure 2. Long-horizon difficulty comes from persistent consequences, fresh context, new evidence and cumulative closure.

Long-horizon signal	What it means	Author check
Work spans real phases	Diagnosis, implementation, review response and hardening are different kinds of work.	Removing a stage would remove meaningful engineering evidence.
State persists	The repository, services and approved outputs survive between stages.	A later stage can observe consequences of earlier work.
Context refreshes	The agent does not keep unlimited conversational memory.	A typed handoff is needed to continue reliably.
New evidence arrives	Review, CI, runtime, compatibility, fault or performance information appears later.	The evidence cannot be fully acted on in Stage 1.
Decisions have consequences	A shortcut in an early stage causes a later failure.	Hidden cases test the same contract under new conditions.
Final closure is cumulative	The last verifier reruns critical requirements from all stages.	A late fix cannot silently break an earlier pass.
Time is useful, not artificial	The task needs investigation, builds, tests, feedback and revision.	Difficulty does not come only from a slow build or oversized timeout.

Deterministic means repeatable

A deterministic check gives the same pass or fail for the same repository state, inputs and environment. It executes behavior directly. It does not ask an LLM judge whether the code seems correct.

2. What the Agent Can and Cannot See

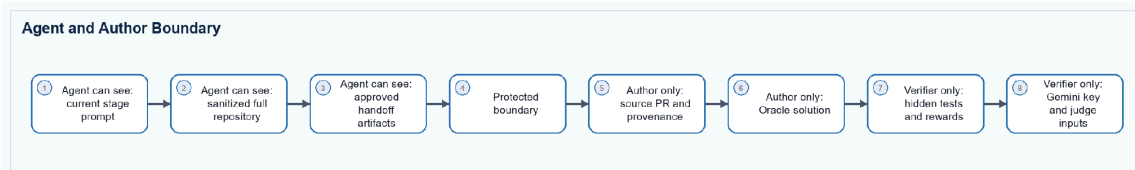


Figure 3. The source change and verifier remain outside the solving environment.

- The source PR is private author evidence. It proves that the work is real and solvable.
- The agent must not receive the PR number, title, future commit, golden patch, hidden tests or solution files.
- The repository image should have one synthetic commit, no origin, no remote refs, no tags and no useful reflog.
- Because the verifier needs internet for the selected RewardKit provider, record and audit any agent network use. Prefer a verifier-only network boundary when the platform supports it.

3. Task Folder Structure

Generic Harbor Package

```

<task-name>/
|-- task.toml                # Harbor metadata, limits, stages, lambda
|-- README.md                # author and operator notes
|-- environment/
|   +-- Dockerfile           # full sanitized repository and pinned tools
|-- tests/                   # protected; not visible during agent work
|   |-- stage_test.sh        # one verifier entrypoint for all stages
|   |-- stage_check          # functional checks and strict reward
|   |-- reward_formula        # deterministic + lambda training score
|   |-- stage_judge_inputs    # creates a small evidence bundle
|   |-- run_rewardkit.sh      # runs the selected LLM judge for agent trials
|   |-- merge_rewardkit_scores # adds training annotations only
|   |-- test_reward_contract  # tests RewardKit discovery and score fields
|   |-- fixtures/            # hidden variants, faults and performance data
|   |-- judge_prompt.md       # shared evidence and authority rules
|   |-- process/judge.toml    # process quality
|   |-- validity/judge.toml   # leakage, cheating and trial validity clues
|   +-- merge_readiness/judge.toml # final patch quality after strict success
|-- steps/
|   |-- 01-diagnose/
|   |   |-- instruction.md
|   |   |-- tests/test.sh
|   |   +-- solution/solve.sh # author-only Oracle for this stage
|   |-- 02-core-implementation/
|   |   |-- instruction.md
|   |   |-- tests/test.sh
|   |   +-- solution/solve.sh
|   |-- 03-review-integration/
|   |   |-- instruction.md
|   |   |-- tests/test.sh
|   |   +-- solution/solve.sh
|   +-- 04-hardening-final/
|       |-- instruction.md
|       |-- tests/test.sh
|       +-- solution/solve.sh

```

Figure 4. Every stage has its own prompt, test entrypoint and author-only Oracle.

```

<task-name>/
|-- task.toml                # Harbor metadata, limits, stages, lambda
|-- README.md                # author and operator notes
|-- environment/
|   +-- Dockerfile           # full sanitized repository and pinned tools
|-- tests/                   # protected; not visible during agent work
|   |-- stage_test.sh        # one verifier entrypoint for all stages
|   |-- stage_check          # functional checks and strict reward
|   |-- reward_formula        # deterministic + lambda training score
|   |-- stage_judge_inputs    # creates a small evidence bundle
|   |-- run_rewardkit.sh      # runs the selected LLM judge for agent trials
|   |-- merge_rewardkit_scores # adds training annotations only
|   |-- test_reward_contract  # tests RewardKit discovery and score fields
|   |-- fixtures/            # hidden variants, faults and performance data
|   |-- judge_prompt.md       # shared evidence and authority rules
|   |-- process/judge.toml    # process quality
|   |-- validity/judge.toml   # leakage, cheating and trial validity clues
|   +-- merge_readiness/judge.toml # final patch quality after strict success
|-- steps/
|   |-- 01-diagnose/
|   |   |-- instruction.md
|   |   |-- tests/test.sh
|   |   +-- solution/solve.sh # author-only Oracle for this stage
|   |-- 02-core-implementation/
|   |   |-- instruction.md
|   |   |-- tests/test.sh
|   |   +-- solution/solve.sh
|   |-- 03-review-integration/
|   |   |-- instruction.md
|   |   |-- tests/test.sh

```

```

| | ++ solution/solve.sh
| | ++ 04-hardening-final/
| | -- instruction.md
| | -- tests/test.sh
| | ++ solution/solve.sh

```

No Python repository tasks

For this task set, choose C/C++, Rust, Go, TypeScript/JavaScript or Java/Kotlin repositories. RewardKit or the protected verifier may use its own harness runtime, but Python is not the repository language being evaluated.

Path	What it is for	Main rule
task.toml	Stages, limits, metadata, model key and lambda	Declare both provider keys and lambda.
environment/Dockerfile	Full repo at the exact base commit	Pin tools and remove source Git memory.
steps/*/instruction.md	The briefing for one stage	State behavior and evidence, not the implementation.
steps/*/tests/test.sh	Small stage verifier launcher	It must exist and call protected functional checks.
steps/*/solution/solve.sh	Author-only Oracle step	Never mount it during an agent phase.
tests/stage_check	All deterministic stage checks	Behavior first; no golden patch matching.
tests/*/judge.toml	Selected RewardKit judge rubrics	Training annotation only.

4. Language Selection Criteria

Choose the repository language after choosing the engineering behavior. The language should make the required runtime contract testable. It should not be used as a shortcut for difficulty.

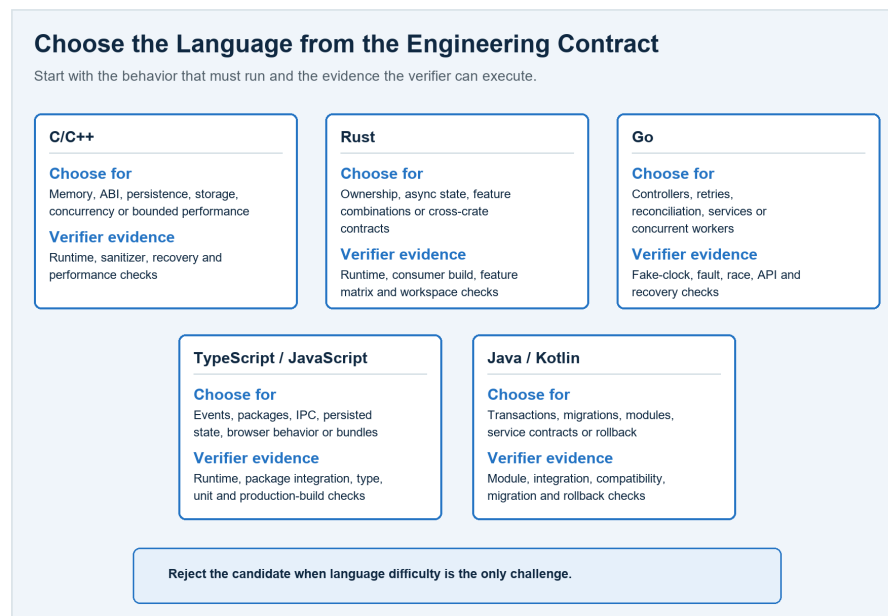


Figure 5. Select the language from the real contract and the deterministic evidence it supports.

C/C++

Best for databases, storage engines, compilers, browsers, media systems, HPC and other repositories where memory, ABI, persistence, concurrency or performance is central.

- Choose when: the task changes ownership, binary layout, indexes, execution state, synchronization, recovery or bounded performance.
- Strong deterministic checks: compile changed targets, execute runtime behavior, run focused regressions, inject faults, check persistence or recovery, and use stable performance thresholds.
- Useful later-stage evidence: sanitizer finding, concurrency schedule, corrupted state, rollback result or workload regression.
- Reject when: the task is only a mechanical rename, a compile fix, or an unbounded flaky stress test.

Rust

Best for runtimes, networking, storage, compilers and security systems where ownership, async state, feature combinations and cross-crate contracts matter.

- Choose when: the change crosses crates, public APIs, caller-owned state, async boundaries, feature flags or safety invariants.
- Strong deterministic checks: focused runtime tests, external-consumer compilation, feature-matrix builds, workspace regressions and fault behavior.
- Useful later-stage evidence: downstream crate failure, feature-specific CI result, async backpressure or compatibility review.
- Reject when: difficulty is only a borrow-checker cascade or a long compile with one obvious edit.

Go

Best for controllers, networking, cloud systems and distributed services where retries, reconciliation, concurrency and operational state can be exercised locally.

- Choose when: the work changes a reconciliation loop, retry policy, API compatibility, concurrent worker or service recovery path.
- Strong deterministic checks: fake clocks, local control planes, bounded race tests, retry/failure injection and API regressions.
- Useful later-stage evidence: stale state, partial service failure, duplicate event, timeout or review about idempotency.
- Reject when: the task is one isolated handler condition with no system consequence.

TypeScript/JavaScript

Best for monorepos, SDKs, event systems, desktop products and web applications where runtime protocols, package boundaries, state and bundling all matter.

- Choose when: the change crosses packages, event streams, IPC, persisted state, browser behavior, build output or public SDK contracts.
- Strong deterministic checks: executable runtime tests, package integration, mocked services or browser state, typecheck, unit tests and production bundle/build checks.
- Useful later-stage evidence: consumer failure, event-order problem, IPC mismatch, bundler error or backward-compatibility review.

- Reject when: typecheck alone can pass the task or tests only search source text.

Java/Kotlin

Best for large services, data platforms, build systems and transaction-heavy applications where modules, migrations, compatibility and recovery matter.

- Choose when: the task crosses modules, transactions, persistence formats, schema versions, service boundaries or rollback behavior.
- Strong deterministic checks: module tests, local integration services, migration and rollback fixtures, compatibility cases and broad regressions.
- Useful later-stage evidence: transaction failure, old-client behavior, migration problem, partial deployment or operational review.
- Reject when: the work is boilerplate API plumbing with no meaningful runtime or data contract.

Language acceptance rule

Before accepting a repository, write down the main behavior, the hidden variants, the regression suite and the final hardening checks. If those cannot be executed reliably, choose a different change or repository.

5. How to Pick a Repository

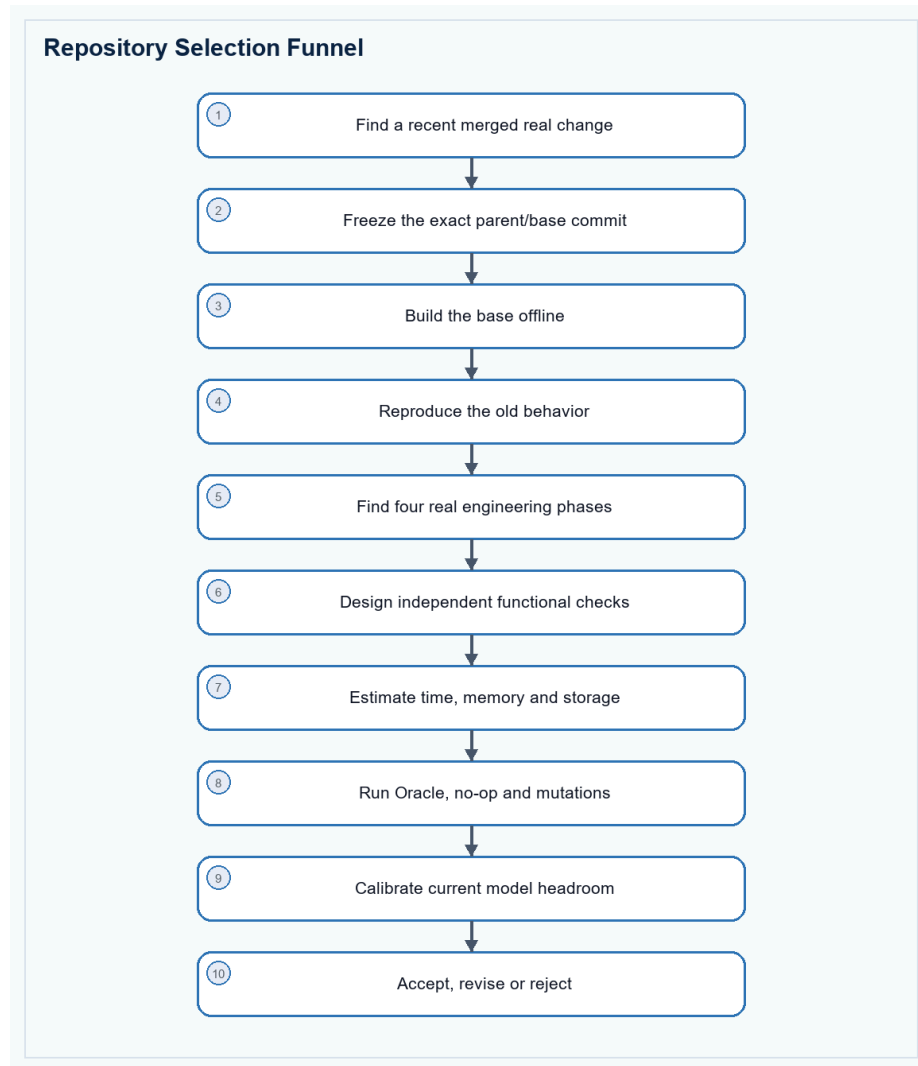


Figure 6. Select for reproducible engineering depth, not line count.

A strong source change has

- A real user, maintainer or production need.
- Repos picked up has to have one of the following licenses:

Apache 2.0, CC BY 4.0, CC BY-SA 4.0, CC BY-NC 4.0, GNU General Public License, BSD 2.0, BSD 3.0 or MIT Licenses

- An exact pre-change commit that still builds and shows the old behavior.
- Several connected contracts: runtime state, data, API, persistence, concurrency, migration, performance or recovery.
- At least three natural engineering phases. Four is preferred.
- More than one valid implementation path.
- Independent hidden cases that are not copied from the public PR.
- A bounded local verifier that can run many times.

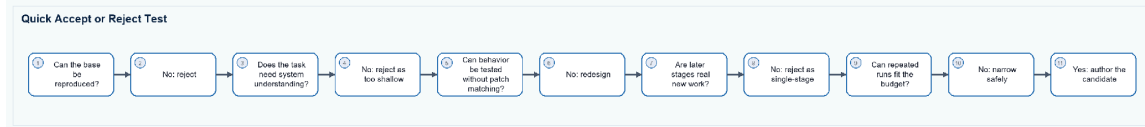


Figure 7. A candidate must pass reproducibility, depth, fairness, stage and cost checks.

Reject or redesign

Candidate	Why it is weak
Docs, formatting or generated files only	Little executable engineering signal.
One obvious function and one visible failing test	Usually too local for Tier 2.
Large mechanical rename	Patch size does not create long-horizon reasoning.
Requires public internet or private services to work	Hard to reproduce and easy to contaminate.
Can pass only by matching exact code or prose	Valid alternative solutions will fail unfairly.
Several prompts but no new evidence after the first stage	It is an artificially split single-stage task.

6. Turn the Real Change into a Benchmark

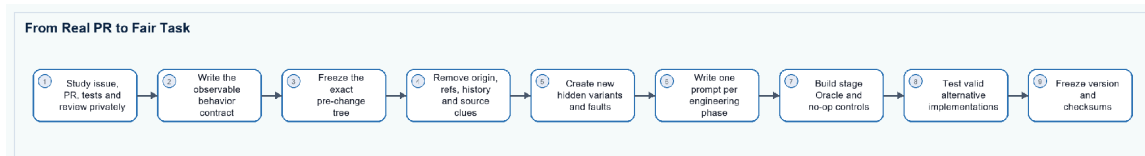


Figure 8. Provenance guides authors but does not become an answer key for the agent.

Golden patch rule

The source PR is not the only accepted solution. The verifier must test required behavior. A different implementation should pass when it is correct, safe and compatible.

7.Flexible Stage Design and Example Flow

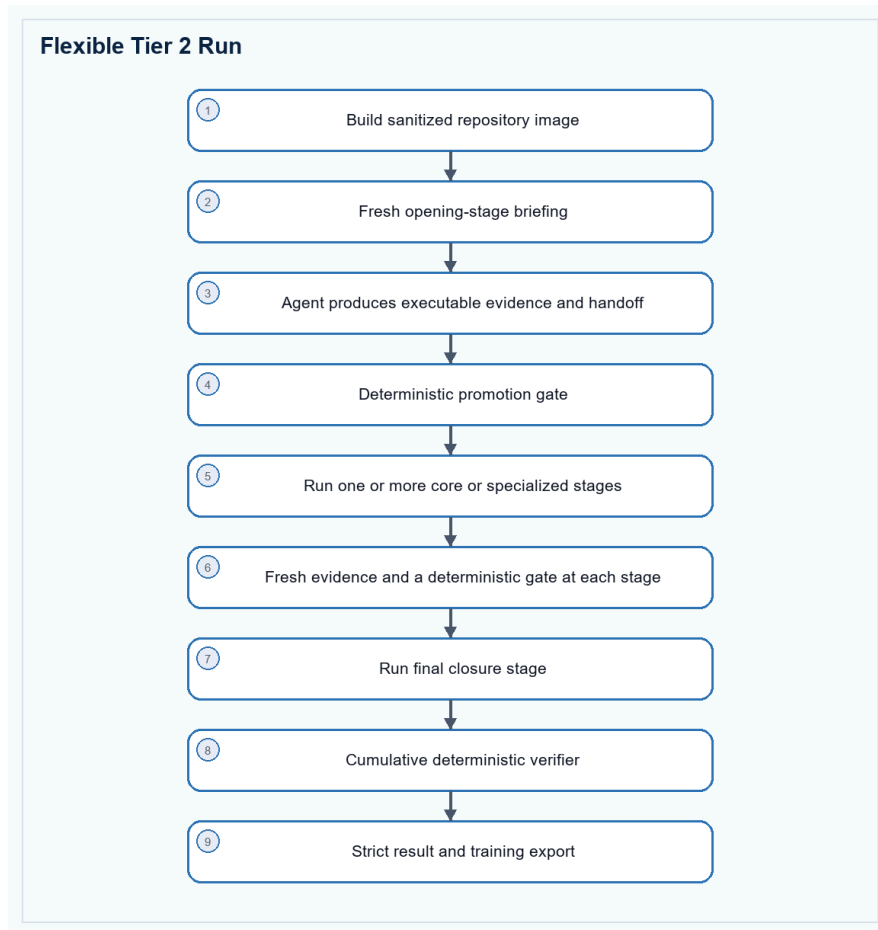


Figure 9. Stage count is task-specific; every stage starts from a fresh briefing while approved repository state and handoffs persist.

Stage role	What the agent may be asked to do	Main deterministic proof	When to include it
Opening evidence	Reproduce, inspect, measure or plan the real problem	Executable evidence, correct scope and an approved handoff	Normally first
Core implementation	Implement the main behavior and focused repository tests	Fail-to-pass behavior, hidden variants, regressions and build health	One or more stages
Integration or feedback	Respond to review, CI, compatibility or downstream evidence	New case plus rechecks of important earlier behavior	When later evidence changes the work
Specialized phase	Handle migration, security, performance, deployment, rollback or recovery	Executable checks for that specific operational contract	Zero or more stages
Final closure	Harden, clean up and prove the accumulated repository state	Cumulative critical checks, regressions, integrity and final artifacts	Always last

Stage count and promotion rule

There is no fixed maximum and four is not mandatory. Normally use at least three meaningful stages: an opening evidence stage, one or more stages that change and integrate the repository, and a final cumulative closure stage. A different count is acceptable when the workflow justifies it. Every non-final promotion gate may use `min_reward = 1.0` when all required functional checks are strict binary; partial progress is still retained for training.

Minimum requirements for every stage

- A clear stage outcome written in the prompt, without revealing the implementation.
- New work or new evidence that was not already completed in the preceding stage.
- At least one executable deterministic check for the main behavior requested in that stage.
- A declared promotion result: pass and continue, or fail with exact reward events.
- A bounded handoff or persistent artifact that the next stage can verify and use.
- An author-only Oracle step that proves this stage is solvable from the promoted prior state.

8. Example Opening Stage - Diagnose and Reproduce

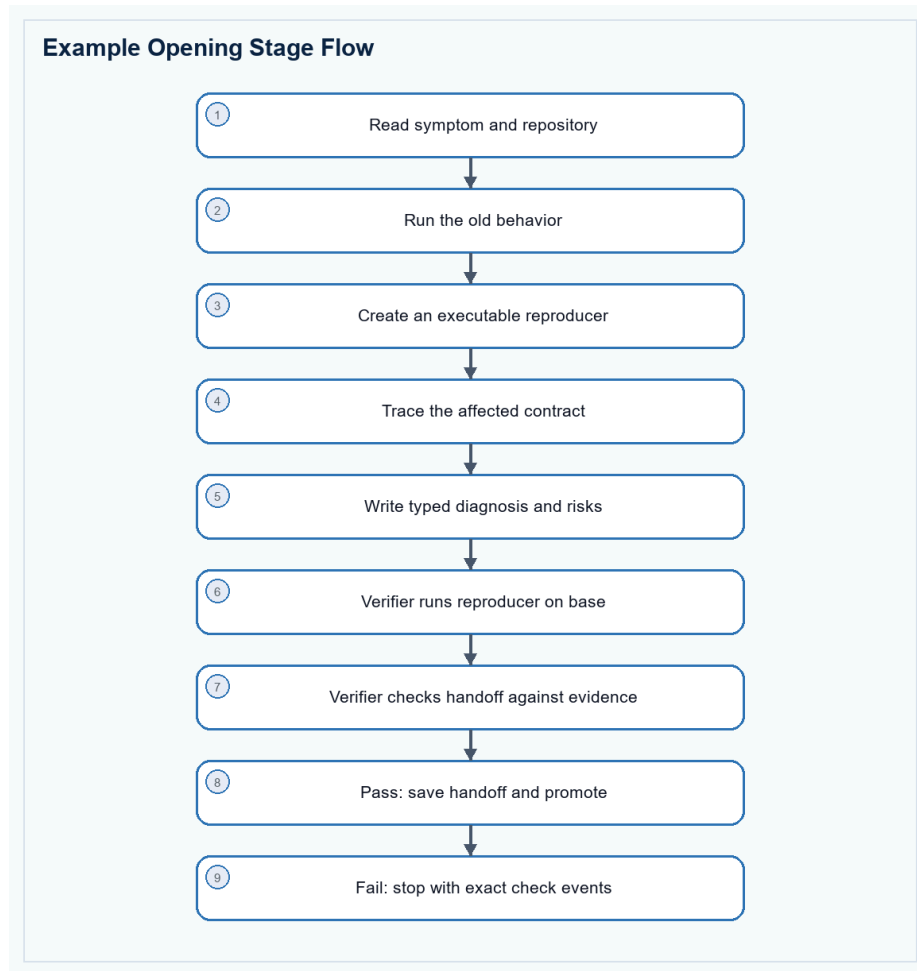


Figure 10. An opening stage can reward proof of understanding rather than a lucky early patch.

Required check	Pass means	Unfair version to avoid
Executable reproducer	It runs the repository and proves the old failure.	Searching the script for one command string.
Observed diagnosis	Typed claims match actual runtime or code evidence.	Requiring exact author wording.
Scope and risk	The handoff names affected contracts and next checks.	Matching an exact golden file list.
Base integrity	No hidden solution, verifier or source history was used.	Using a judge guess as proof of cheating.
Build health	The base remains usable after diagnosis work.	Demanding a full expensive build when a documented narrow build is enough.

9. Example Core Stage - Implementation

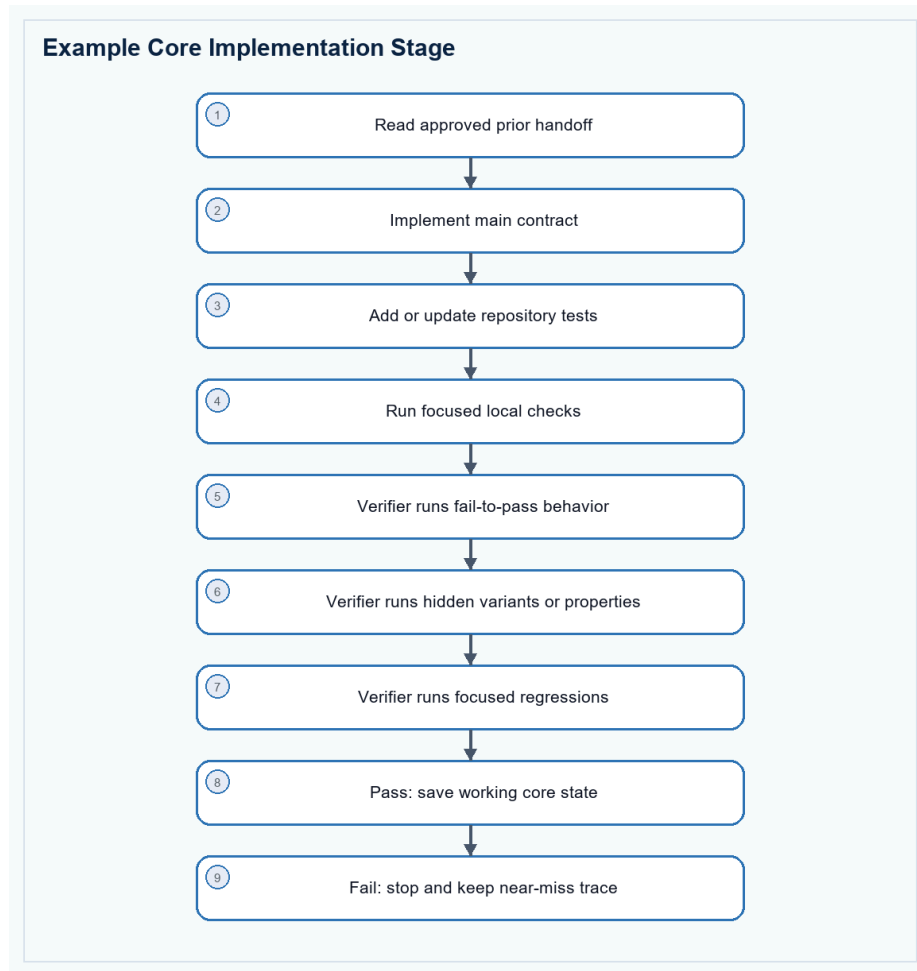


Figure 11. A core stage tests the requested behavior and nearby regressions.

Check family	Typical weight	What it should test
Primary behavior	40-50%	The exact user-visible or API behavior requested.
Hidden variants or properties	20-25%	Different data shapes, orders, states or inputs.
Focused pass-to-pass	15-20%	Existing behavior close to the change.
Build or typecheck	5-10%	The changed targets remain usable.
Handoff and evidence	5-10%	The agent used approved prior evidence and reports real commands.

10. Example Intermediate Stage - Review and Integration

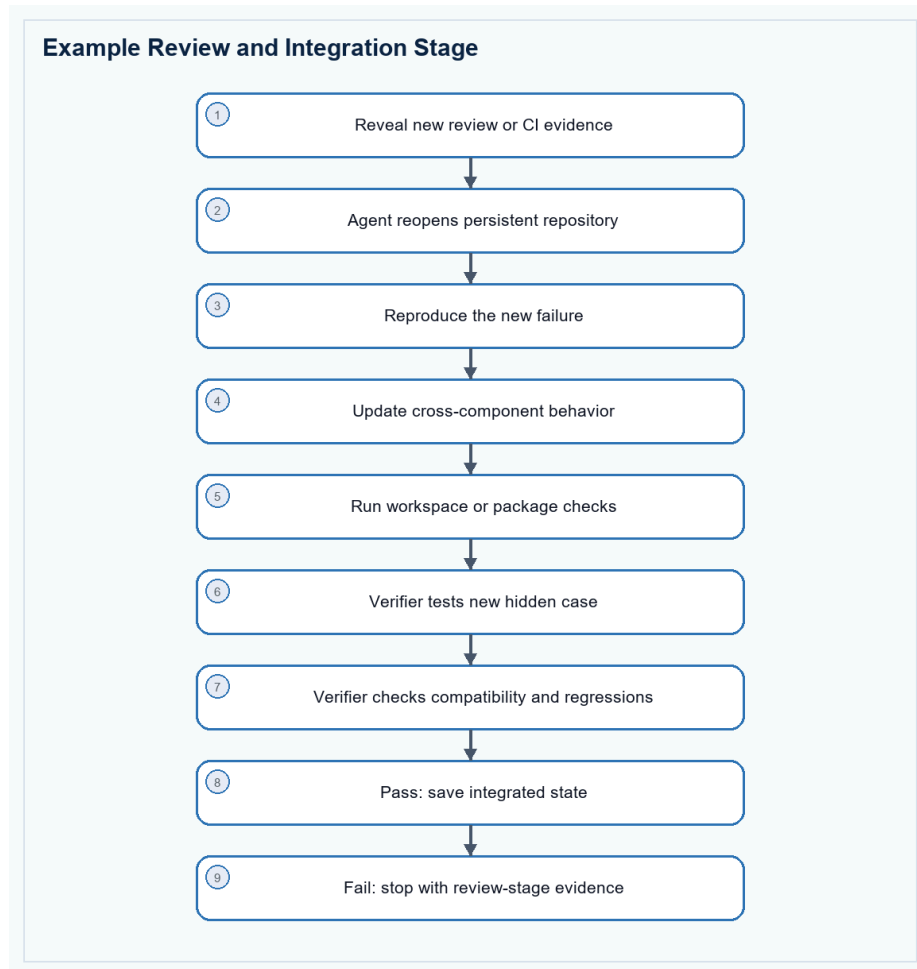


Figure 12. An intermediate stage can test adaptation to new evidence without breaking earlier work.

- The feedback must add a real requirement, downstream failure or compatibility case.
- The prompt must show enough evidence for the agent to reproduce the issue.
- The verifier must rerun the important earlier core behavior as well as the new integration case.
- A prose review response earns credit only when it matches repository changes and executable output.

11. Example Final Stage - Hardening and Closure

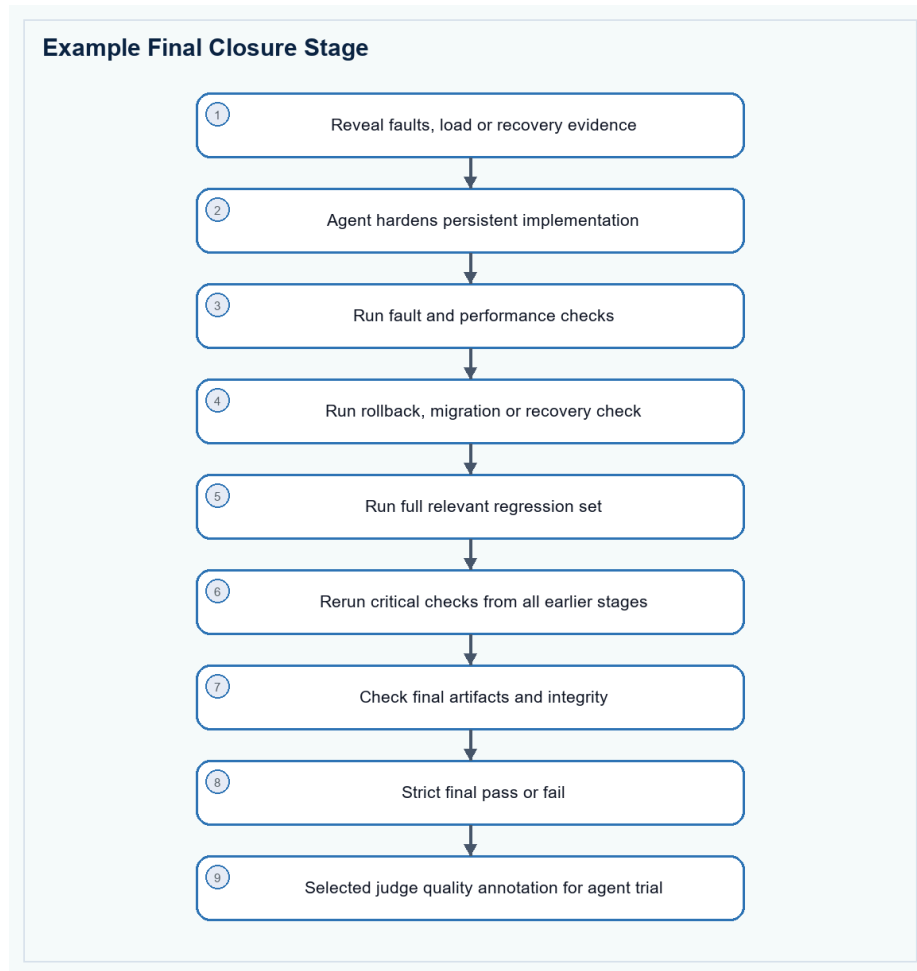


Figure 13. A final stage tests robustness and rechecks the whole accumulated change.

Hardening area	Good deterministic check	Bad check
Fault handling	Inject a real bounded failure and assert recovery or safe error behavior.	Search source for error-handling words.
Performance	Compare a stable workload with a fixed threshold and warmup.	Fail on a noisy single timing.
Migration	Run forward and rollback paths against real fixture data.	Check that a migration filename exists.
Concurrency	Use controlled schedules, barriers or repeated seeds.	Rely on an unbounded flaky stress test.
Final closure	Rerun all critical earlier behavior and regressions.	Trust the previous stage result without rechecking.

12. How Deterministic Checks Work

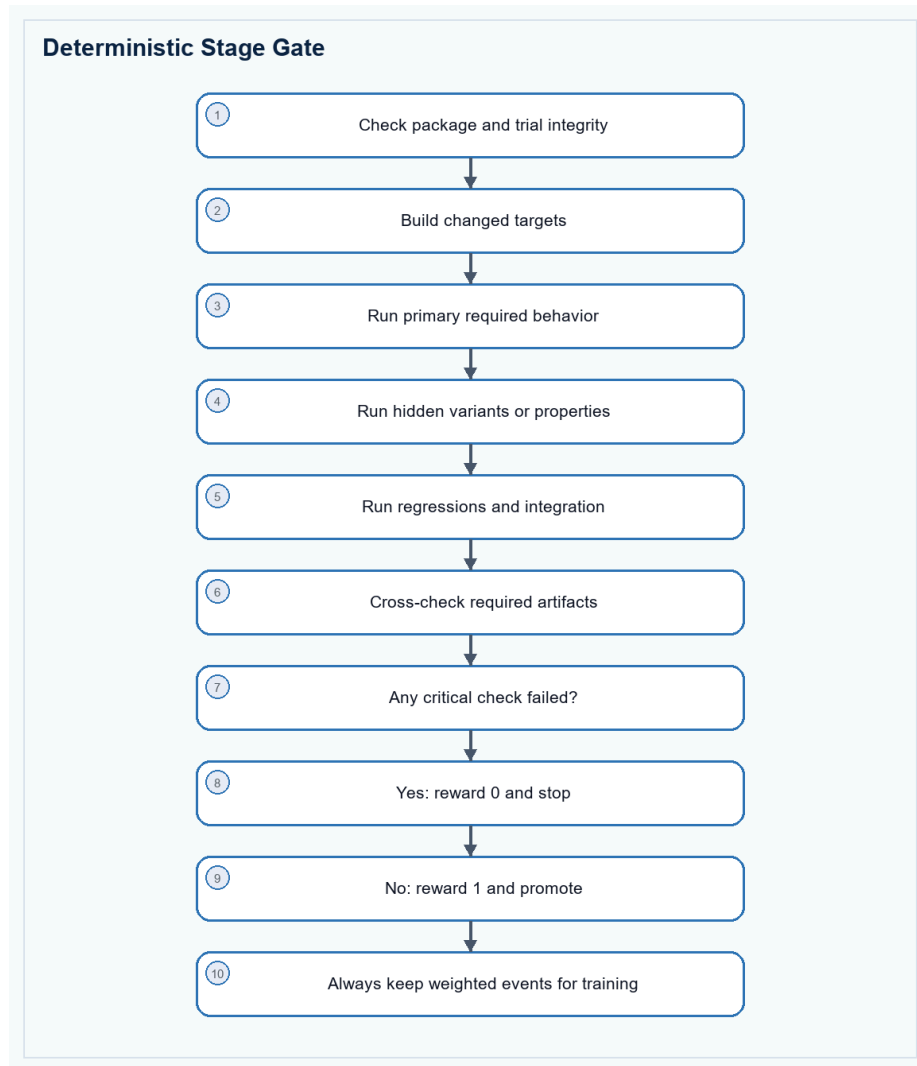


Figure 14. Required behavior decides promotion; partial weights explain progress.

Check group	Suggested share	Design rule
Primary required behavior	35-50%	The main user-visible contract; critical
Hidden variants or properties	15-25%	Reject fixture-specific and hard-coded fixes
Pass-to-pass regressions	10-20%	Existing behavior stays correct
Integration or data correctness	10-20%	The change works across real boundaries
Build, typecheck or package	5-10%	The repository remains usable
Artifact and handoff evidence	5-10%	Claims match executable results
Integrity	Critical gate	No verifier, solution, history or reward tampering

Functionality first

Use real commands, APIs, services, files, databases and outputs. Avoid exact patch, exact AST, exact file-count and exact prose matching unless that exact form is itself a public requirement.

13. Reward, Lambda and Training Score

Deterministic evaluation and lambda-weighted training

Deterministic progress

$$D_score = \sum_i (u_i \times w_i) / \sum_i w_i$$

LLM rubric progress

$$N_score = \sum_j (r_j \times v_j) / \sum_j v_j$$

Official deterministic evaluation reward

reward = binary_reward = 1 only if valid_trial = 1
and every required deterministic check passes; otherwise 0

Normalized training score

$$T_lambda = [\sum_i (u_i \times w_i) + \lambda \sum_j (r_j \times v_j)] / [\sum_i w_i + \lambda \sum_j v_j]$$

u_i: deterministic result w_i: deterministic weight r_j: LLM rubric score v_j: rubric weight

 $\lambda = 0 \Rightarrow \text{training_score} = D_score$

The denominators keep deterministic_score, LLM score and training_score between 0 and 1.

Figure 15. Deterministic checks control binary evaluation; normalized LLM rubrics enter only the lambda-weighted training score.

```

For deterministic check i:
    u_i = check result in [0, 1]
    w_i = deterministic weight

D_num = sum(u_i * w_i)
D_den = sum(applicable w_i)
D_score = D_num / D_den

Official evaluation reward:
    reward = binary_reward = 1
            only when valid_trial = 1 and every required deterministic check passes;
            otherwise reward = binary_reward = 0.

For LLM rubric j:
    r_j = rubric score in [0, 1]
    v_j = rubric weight

N_num = sum(r_j * v_j)
N_den = sum(applicable v_j)
N_score = N_num / N_den

Training score:
    training_score =
        (D_num + lambda * N_num) /
        (D_den + lambda * N_den)

If lambda = 0 or no LLM rubric is applicable:
    training_score = D_score

```

Symbol or field	Meaning	Rule
u_i	Result of deterministic check i	0 or 1 for a binary test; may be 0..1 only for a deliberately partial deterministic measure
w_i	Weight of deterministic check i	Give the main requested behavior the largest weight
D_score	Normalized deterministic progress	D_num / D_den , always 0..1
reward / binary_reward	Official evaluation result	1 only if the trial is valid and every required deterministic check passes; otherwise 0
r_j	Score from LLM rubric j	0..1; process or quality only, never functional truth
v_j	Weight of LLM rubric j	Applied only when that rubric is applicable
λ	Strength of LLM evidence in training score	0 gives deterministic-only training; normally keep in 0..1
training_score	Normalized deterministic plus LLM training signal	$(D_num + \lambda * N_num) / (D_den + \lambda * N_den)$, always 0..1

Field	Meaning	Range or value
reward	Official strict stage or final result	0 or 1
binary_reward	Same strict benchmark success signal	0 or 1
deterministic_score	Weighted deterministic progress	0 to 1
non_deterministic_score	Selected LLM judge annotation when applicable	0 to 1 or N/A
training_score_lambda	How much judge evidence affects training score	0 to 1 for this policy; N/A for Oracle
training_score	Normalized blend for training and analysis	0 to 1
valid_trial	Whether the run is usable and untampered	0 or 1

What lambda does

- $\lambda = 0$: training_score is exactly the deterministic score. LLM judge output can still be stored, but it has zero score weight.
- $\lambda = 0.5$: judge evidence has half its normal denominator weight. The formula still stays between 0 and 1.
- $\lambda = 1$: deterministic and judge denominators are combined at full declared weights.
- λ never changes reward, binary_reward, stage_complete, promotion or validity.
- If no judge result is applicable, use only the deterministic denominator. Do not treat a missing judge as zero quality.

Agent reward example

```
{
  "reward": 0.0,
  "binary_reward": 0.0,
  "valid_trial": 1.0,
  "stage_complete": 0.0,
  "deterministic_score": 0.76,
  "deterministic_numerator": 7.6,
  "deterministic_denominator": 10.0,
  "non_deterministic_score_applicable": 1.0,
  "non_deterministic_score": 0.60,
  "non_deterministic_numerator": 2.4,
  "non_deterministic_denominator": 4.0,
```

```
"training_score_lambda": 0.5,  
"training_score": 0.733  
}
```

Oracle reward example

```
{  
  "reward": 1.0,  
  "binary_reward": 1.0,  
  "valid_trial": 1.0,  
  "stage_complete": 1.0,  
  "deterministic_score": 1.0,  
  "non_deterministic_score_applicable": 0.0,  
  "non_deterministic_score": null,  
  "training_score_lambda": null,  
  "merge_readiness": null,  
  "training_score": 1.0  
}
```

Oracle rule

The Oracle proves deterministic solvability. Do not run merge-readiness or process judges for Oracle. Judge-only fields and training_score_lambda are N/A, shown as null in machine output.

14. RewardKit with Sonnet or Gemini 3.6 Flash

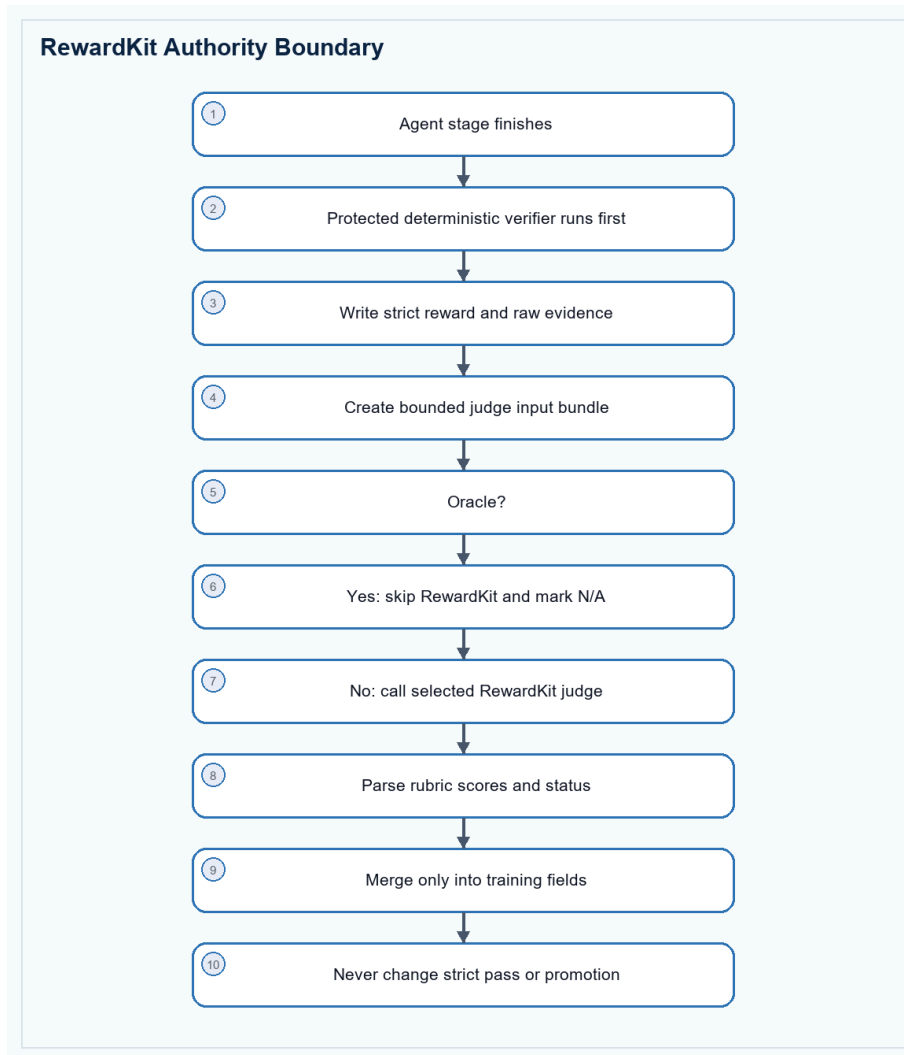


Figure 16. RewardKit runs after deterministic truth and has limited authority.

Provider keys and lambda in task.toml

```
[verifier.env]
ANTHROPIC_API_KEY = "${ANTHROPIC_API_KEY:-}"
GEMINI_API_KEY = "${GEMINI_API_KEY:-}"
NONDETERMINISTIC_LAMBDA = "${NONDETERMINISTIC_LAMBDA:-0.0}"
```

Declare both provider keys and lambda in verifier.env. Choose one RewardKit judge in judge.toml and use the same choice across all judge files. If both keys are exported, judge.toml still decides which provider runs. Oracle is detected by the trusted run mode and always skips LLM judging.

Stage process and validity judge

Option A - Sonnet 4.6 judge

```
[[judge]]
judge = "anthropic/claude-sonnet-4-6"
prompt_template = "../judge_prompt.md"
files = ["/logs/verifier/judge_inputs"]

[[criterion]]
name = "evidence_based_process"
description = '''
Use only the current prompt, bounded trajectory, patch, typed artifacts and
raw deterministic results. Score whether the agent investigated, tested and
responded to evidence well. Do not replace executable correctness checks.
'''
type = "continuous"
weight = 0.55

[[criterion]]
name = "validity_and_anti_cheating"
description = '''
Look for likely leakage, hidden-test access, reward hacking, fabricated
evidence, prompt mismatch or infrastructure failure. This is a review flag.
It cannot fail or pass a stage without independent confirmation.
'''
type = "continuous"
weight = 0.45
```

Option B - Gemini 3.6 Flash judge

```
[[judge]]
judge = "gemini/gemini-3.6-flash"
prompt_template = "../judge_prompt.md"
files = ["/logs/verifier/judge_inputs"]

[[criterion]]
name = "evidence_based_process"
description = '''
Use only the current prompt, bounded trajectory, patch, typed artifacts and
raw deterministic results. Score whether the agent investigated, tested and
responded to evidence well. Do not replace executable correctness checks.
'''
type = "continuous"
weight = 0.55

[[criterion]]
name = "validity_and_anti_cheating"
description = '''
Look for likely leakage, hidden-test access, reward hacking, fabricated
evidence, prompt mismatch or infrastructure failure. This is a review flag.
It cannot fail or pass a stage without independent confirmation.
'''
type = "continuous"
weight = 0.45
```

Final merge-readiness judge

```
[[judge]]
# Choose exactly one model and use it in every judge.toml file.
judge = "anthropic/claude-sonnet-4-6"
# judge = "gemini/gemini-3.6-flash"
prompt_template = "../judge_prompt.md"
files = ["/logs/verifier/judge_inputs"]

[[criterion]]
name = "merge_readiness"
description = '''
Run only for a valid agent trial. Assess repository conventions, focus,
maintainability, architecture, useful tests, operability and patch bloat.
This score is a training annotation and never changes strict success.
'''
type = "continuous"
weight = 1.0
```

LLM judge review	When it runs	What it may do	What it may not do
Process	Every agent stage	Score investigation, evidence and verification discipline.	Decide functional correctness.
Validity and anti-cheating	Every agent stage	Flag likely leakage, hacking, ambiguity or infrastructure.	Invalidate by itself.
Merge readiness	Final valid agent stage	Score focus, conventions, maintainability and bloat.	Rescue a deterministic failure.

15. task.toml and Run Commands

```

schema_version = "1.3"
multi_step_reward_strategy = "final"

[task]
name = "long-horizon/<task-slug>"
description = "One real repository change completed over four engineering stages."

[metadata]
benchmark_type = "swe-bench"
category = "software-engineering"
language = "<cpp|rust|go|typescript|javascript|java|kotlin>"
tier = "stateful-long-horizon-engineering"
repository = "<owner/repository>"
base_commit = "<exact-pre-change-commit>"
source_pull_request = "<author-only URL>"
rewardkit_judge_options = "anthropic/claude-sonnet-4-6 | gemini/gemini-3.6-flash"
reward_formula = "strict deterministic binary reward"
training_score_formula = "(D_num + lambda * N_num) / (D_den + lambda * N_den)"

[environment]
build_timeout_sec = 7200.0
allow_internet = true
cpus = 8
memory_mb = 16384
storage_mb = 65536

[verifier.env]
ANTHROPIC_API_KEY = "${ANTHROPIC_API_KEY:-}"
GEMINI_API_KEY = "${GEMINI_API_KEY:-}"
NONDETERMINISTIC_LAMBDA = "${NONDETERMINISTIC_LAMBDA:-0.0}"

[[steps]]
name = "01-diagnose"
min_reward = 1.0
artifacts = ["/app/output/01-diagnose", "/app/output/handoff.json"]
[steps.agent]
timeout_sec = 7200.0
[steps.verifier]
timeout_sec = 1800.0

[[steps]]
name = "02-core-implementation"
min_reward = 1.0
artifacts = ["/app/output/02-core", "/app/output/handoff.json"]
[steps.agent]
timeout_sec = 14400.0
[steps.verifier]
timeout_sec = 3600.0

[[steps]]
name = "03-review-integration"
min_reward = 1.0
artifacts = ["/app/output/03-review"]
[steps.agent]
timeout_sec = 10800.0
[steps.verifier]
timeout_sec = 3600.0

[[steps]]
name = "04-hardening-final"
artifacts = ["/app/output/04-hardening"]
[steps.agent]
timeout_sec = 10800.0

```

```
[steps.verifier]
timeout_sec = 5400.0
```

Internet setting

allow_internet is true here so the verifier can call the selected RewardKit provider. If Harbor cannot change network policy between agent and verifier, treat agent internet access as a known risk: audit commands and traffic, remove source identifiers and use novel hidden variants.

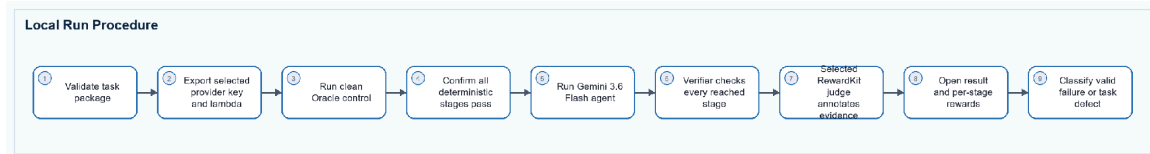


Figure 17. Run deterministic controls first, then the agent and selected RewardKit judge.

Oracle: deterministic control. RewardKit is skipped and judge fields are N/A.

```
harbor run \
-p ./<task-name> \
-a oracle \
--jobs-dir /root/harbor-jobs \
--job-name <task>-oracle-v1 \
--yes
```

The solving agent remains Gemini 3.6 Flash.
export GEMINI_API_KEY="<google-ai-studio-key>"
export NONDETERMINISTIC_LAMBDA="0.0"

Option A: Sonnet 4.6 is selected in every judge.toml.
export ANTHROPIC_API_KEY="<anthropic-key>"

```
harbor run \
-p ./<task-name> \
-a terminus-2 \
-m gemini/gemini-3.6-flash \
--jobs-dir /root/harbor-jobs \
--job-name <task>-gemini-agent-sonnet-judge-v1 \
--ve GEMINI_API_KEY="$GEMINI_API_KEY" \
--ve ANTHROPIC_API_KEY="$ANTHROPIC_API_KEY" \
--ve NONDETERMINISTIC_LAMBDA="$NONDETERMINISTIC_LAMBDA" \
--yes
```

Option B: Gemini 3.6 Flash is selected in every judge.toml.

```
harbor run \
-p ./<task-name> \
-a terminus-2 \
-m gemini/gemini-3.6-flash \
--jobs-dir /root/harbor-jobs \
--job-name <task>-gemini-agent-gemini-judge-v1 \
--ve GEMINI_API_KEY="$GEMINI_API_KEY" \
--ve NONDETERMINISTIC_LAMBDA="$NONDETERMINISTIC_LAMBDA" \
--yes
```

Read results in this order

File	What to check
result.json	Terminal stage, exception, reward, binary_reward and valid_trial.
steps/*/verifier/reward.json	Strict stage result and score applicability.
reward-details.json	Exact deterministic check IDs, weights and evidence.

File	What to check
test_output.log	Whether a check really ran or the verifier crashed.
reward-events.json	Granular deterministic and LLM-judge training events.
judge_status.json	Selected provider call, parser result and any unavailable rubric.
trajectory.json and patch.diff	Engineering process, changed files, network use and tampering clues.

How to Read a Failure

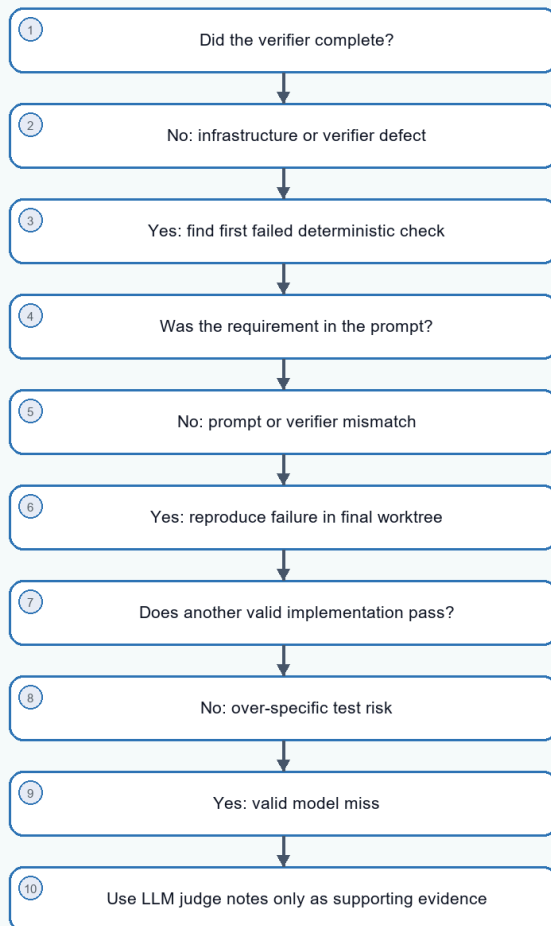


Figure 18. A model failure is accepted only after basic fairness checks.

16. Oracle, No-Op and Near-Miss Tests



Figure 19. Controls prove that the verifier rewards complete behavior and rejects realistic shortcuts.

Control	Expected result	If it does not happen
Oracle repeated at least three times	Every deterministic stage and final closure pass.	Fix solution, environment, transition or verifier.
Clean no-op	Fails the first real behavior.	The base is already solved or checks are weak.
Compile-only patch	Fails runtime behavior.	Build credit is too generous.
Partial patch	Fails the exact missing requirement.	Critical checks or weights are wrong.
Visible-fixture hard code	Fails hidden/property variants.	Verifier is overfit.
Regression patch	Fails pass-to-pass or integration.	Regression coverage is weak.
Valid alternative patch	Passes all behavior.	A failure suggests golden-patch matching.

17. Anti-Cheating

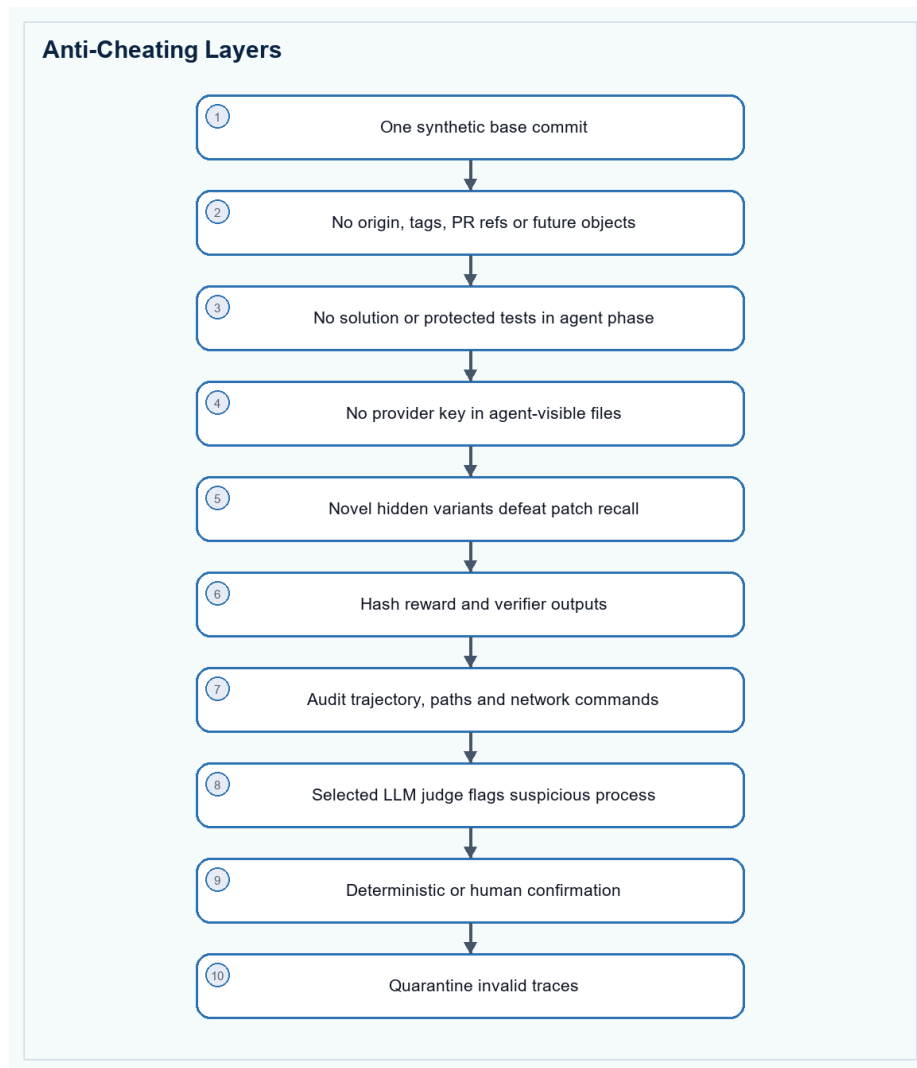


Figure 20. Anti-cheating uses several independent layers.

- A patch that resembles the real PR is not proof of cheating. Correct solutions often share the same idea.

- Look for reachable source history, public identifiers, network lookup, hidden-test access, solution access, reward-file edits or hard-coded hidden cases.
- The selected LLM judge can flag suspicious behavior, but a flag needs command, file, network or deterministic evidence before invalidation.
- Store contamination and validity labels separately from model capability labels.

18. Training Export

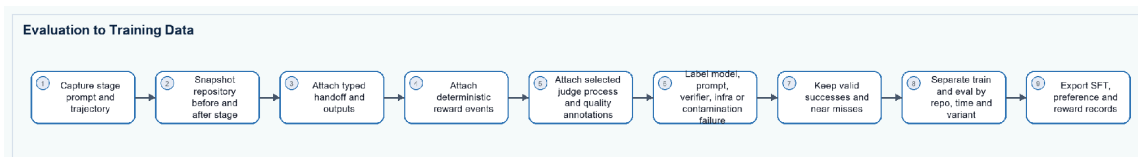


Figure 21. Keep the temporal story and the reason for each outcome.

Export	Training use	Rule
Stage trajectories	Planning, tool use, diagnosis and recovery.	Remove secrets and protected verifier text.
Before/after snapshots	Learning state transitions over time.	Hash and bind to exact task version.
Deterministic events	Dense outcome and repair signals.	Keep stable check IDs and raw evidence.
Valid near misses	Contrastive and repair training.	Exclude unfair or infrastructure failures.
LLM judge annotations	Process and merge-quality preferences.	Store model name and keep separate from truth.
Oracle traces	Positive demonstrations.	Never mix the exact eval task into training.

19. Release and Headroom

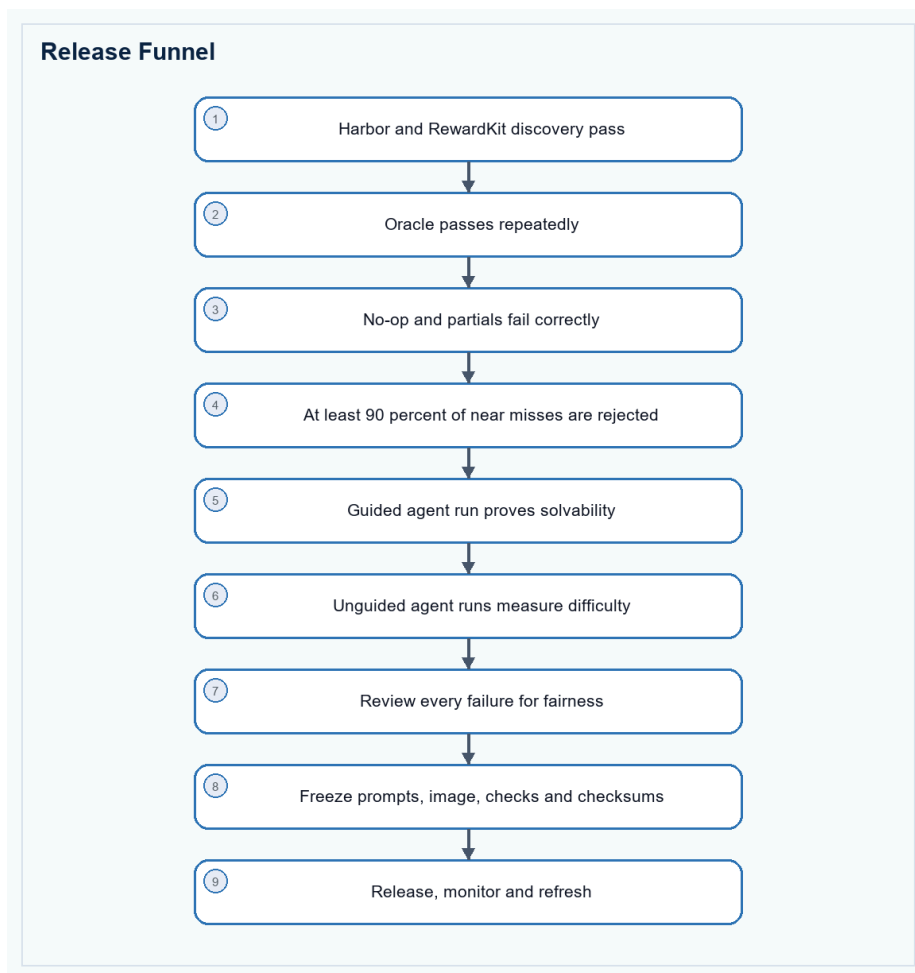


Figure 22. Release needs solvability, fairness and useful model headroom.

Gate	Minimum result
Package	Harbor finds every stage test; RewardKit discovery and parser tests pass.
Oracle	Three clean repeated runs pass all deterministic stages.
No-op	Always fails the first intended behavior.
Near misses	At least 90% fail the intended check family.
Fairness	Valid alternative solutions pass; no hidden prose requirement.
Long horizon	At least three real phases, persistent state and new later evidence.
Headroom	Aim for roughly 5-20% strict solve rate on current fixed Gemini runs.
Scoring	Strict reward is binary; all training scores stay in 0 to 1.
Split	Training and evaluation differ by repository, time window, variant and checksum.

20. Compact Build Checklist

- Choose a recent merged change in C/C++, Rust, Go, TypeScript/JavaScript or Java/Kotlin.
- Freeze and reproduce the exact pre-change commit.
- Prove that diagnosis, core work, review integration and hardening are real phases.
- Use the full sanitized repository with one synthetic commit and no source history.
- Write one clear prompt per stage and disclose every strict artifact contract.
- Give the main required behavior the most deterministic weight.
- Use executable hidden variants, regressions, integration, faults and recovery checks.
- Keep reward and binary_reward strict 0 or 1.
- Use the normalized lambda formula only for training_score.
- Use gemini/gemini-3.6-flash for the solving agent. Choose either anthropic/claude-sonnet-4-6 or gemini/gemini-3.6-flash for all RewardKit judge files.
- Skip RewardKit for Oracle and mark judge-only fields N/A.
- Run Oracle, no-op, near-miss, alternative-solution and fairness controls before release.

21. Source Map

Public design references were checked on July 28, 2026. Source PR details belong in protected author records, not in the agent prompt.

Source	URL	Used for
Harbor multi-step tasks	harborframework.com	Ordered stages, persistent environment and artifacts
Harbor RewardKit	harborframework.com	Judge package and verifier integration
SWE-bench	swebench.com	Full-repository behavioral evaluation
Senior SWE-Bench	senior-swe-bench.snorkel.ai	Realistic recent repository task design